表面筋電位解析のための on-device 推論・学習システムの開発と応用評価

大阪公立大学工業高等専門学校 安藤研究室
学籍番号 R21094
氏名 年澄 荘多

令和 7 年度

目次

第 1 章　はじめに 1
1.1　研究背景 1
1.2　研究目的 1

第 2 章　関連研究 2
2.1　既存研究における sEMG ジェスチャ分類 2
2.2　本研究の位置づけ 2

第 3 章　システム構成およびデータ収集 3
3.1　システム全体アーキテクチャ 3
3.2　ハードウェア構成 4
3.3　ソフトウェア構成 5

第 4 章　モデル設計および学習手法 12
4.1　特徴量設計 12
4.2　ニューラルネットワーク構成 13

第 5 章　実験および評価 15
5.1　実験条件 15
5.2　評価指標 16
5.3　実験結果 18

第 6 章　考察 21
6.1　結果の考察 21
6.2　課題 22

第 7 章　結論 23

参考文献 24

謝辞 25

付録 A-1
付録 A：特徴量の詳細な数式定義 A-1

第 1 章

はじめに

1.1 研究背景

表面筋電位（surface electromyography：sEMG）は, 筋活動を非侵襲的に計測可能な生体信号であり, ヒューマンインタフェースなどへの応用が期待されている. 一方で,sEMG 信号はノイズや装着条件, 個人差の影響を受けやすく, 特に単一チャネル構成では安定した分類性能の確保が課題となる.

近年, モバイル端末の高性能化により, エッジデバイス上でリアルタイムに動作する sEMG 解析システムへの需要が高まっている. さらに, 利用者固有のデータを用いてモデルを更新する on-device 学習は, 個人差や環境変動への適応手法として注目されているが, モバイル端末では計算資源が限られていることや関連ライブラリの未成熟から実装難度が高い. そのため実システムにおける sEMG 分類に関する on-device 学習の評価例は限られている.

1.2 研究目的

本研究の目的は, 単一チャネル sEMG を対象とした on-device 推論・学習システムを開発し,on-device 学習がリアルタイムジェスチャ検出の実用性能に与える影響を評価することである.

第 2 章

関連研究

2.1 既存研究における sEMG ジェスチャ分類

表面筋電位（surface electromyography：sEMG）分類では, 信号取得後に特徴量を抽出し, 分類器によってジェスチャを識別する手法が一般的である.

Arozi ら [1] は, 単一チャネル sEMG に対して時間領域特徴量と人工ニューラルネットワーク（ANN）を用いることで, 複数の手指動作を分類可能であることを示した. また,Ibrahim ら [2] は, 単一チャネル sEMG を用いた分類器をマイクロコントローラ上に実装し, リアルタイム推論の実現性を示した. 一方で, これらの研究はいずれも事前学習モデルを使用しており, 動的に変化する個人差や環境変動への適応は検討されていない. 実運用では電極の装着条件や個人差などの動的な条件変化が発生することは避けられず, これらを静的に解決するためには推論時の計算資源や, 事前の開発 (学習) コストが大きくなる.

2.2 本研究の位置づけ

以上の先行研究では, 単一チャネル sEMG を対象とした軽量な分類器の構築や, エッジデバイス上でのリアルタイム推論実装が主に検討されてきた. 一方で, 実運用を想定し, モデルの学習までを含めて on-device で適応させる手法や, その効果を評価した研究は限られている.

本研究では, 先行研究で提案されている複数の特徴量設計およびモデル構成について, 自身の研究で用いるハードウェアおよびソフトウェア環境において再現実験を行った. その結果, 時間領域特徴量 10 次元（AR 係数 4 次元を含む）をそのまま入力とし,1 層の全結合層を介して 4 クラスを出力する ANN 構成が, 分類精度および処理速度の両面で最も優れていることを確認した.

以上を踏まえ, 本研究ではこの軽量な ANN モデルを採用し, モデルを更新可能な構成として on-device 学習による分類精度および実運用性能の向上を検証する. 既存研究で主に扱われてきた推論の on-device 化に加え, 学習を含めた on-device 化に着目する点が本研究の特徴である.

第 3 章

システム構成およびデータ収集

3.1　システム全体アーキテクチャ

本システムは, 大別して 3 層からなるアーキテクチャで構成される（図 3.1）.

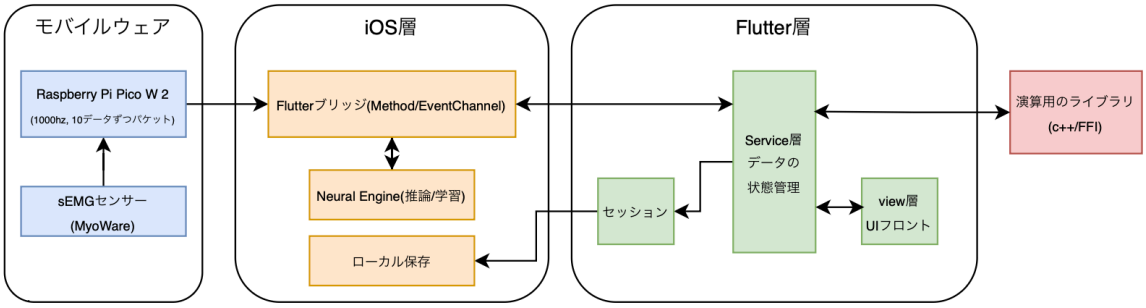


図 3.1　システム全体アーキテクチャ

3.1.1　全体構成

1. **モバイルウェア層（ペリフェラルデバイス）**
   Raspberry Pi Pico 2 W と MyoWare センサーで構成される.sEMG の有効周波数帯域は 20–500Hz とされており [3], サンプリング定理を満たすため 1kHz で ADC 変換を行う. このサンプリングレートを BLE Notify の制約内で伝送するため,10 サンプルずつパケット化して Notify する.
2. **iOS 層（ネイティブ実行環境）**
   Flutter ブリッジが Method Channel / Event Channel を介して Flutter 層と通信し,Neural Engine（Core ML）を用いた推論・on-device 学習, ローカルストレージへのセッション永続化を担当する.
3. **Flutter 層（アプリケーション層）**

BLE 通信, データバッファリング, セッション管理を行う Service 層,UI を提供する View 層, 高速演算を担う C++/FFI ライブラリ（特徴抽出）から構成される.

3.2 ハードウェア構成

Raspberry Pi Pico 2 W をマイコンとして使用し,sEMG 信号を BLE 経由でモバイル端末へ送信する.

3.2.1 ハードウェアの選定

表 3.1 ハードウェア構成一覧

コンポーネント	型番/仕様	用途
マイコン	Raspberry Pi Pico 2 W	信号サンプリング・BLE 通信
sEMG センサー	MyoWare 2.0 筋電位センサー	表面筋電位計測
開発環境	MicroPython	ファームウェア実装

3.2.2 ファームウェア（Raspberry Pi Pico 2 W）

MicroPython ベースの BLE Peripheral 実装により, 以下の機能を実装する.

- **サンプリング**：内蔵 ADC4 を用いた 1kHz 連続サンプリング
- **パケット** ：シーケンス番号（2 バイト）+ 10 サンプル（各 4 バイト float）の 42 バイト
- **送信レート**：100Hz（10 パケット周期）で Notify 配信

3.3 ソフトウェア構成

3.3.1 技術選定

メインアプリケーションは Google 社が開発したフレームワークである Flutter を採用し, クリーンアーキテクチャに基づいて各処理を Service として定義し,Riverpod で状態管理する（図 3.2）. Flutter は単一コードベースで iOS/Android アプリを同時開発できる. 本研究は現時点で iOS 向けの実装を行ったが, 後述する Service 単位でのロジック分離により,Android 対応への拡張が容易な設計となっている. BLE 通信, データバッファリング, 特徴抽出などはプラットフォーム非依存の Dart コードと C++ コンパイルバイナリの FII 呼び出しで実装し, 機械学習推論部分のみを Method Channel 経由で Native 層に委譲している. Android 対応時には TensorFlow Lite 実装を Kotlin で追加することで,Flutter 層を変更せずに Android 版を構築できる. 本システムで使用する主要技術およびライブラリを表 3.2 に示す.

表 3.2　ソフトウェア構成一覧

カテゴリ	技術/ライブラリ	用途
フレームワーク	Flutter 3.x	クロスプラットフォーム UI
言語（Flutter 層）	Dart	アプリケーションロジック
言語（Native 層）	Swift	iOS Native 機能実装
言語（演算処理）	C++	特徴抽出の高速演算
状態管理	Riverpod	依存性注入・状態管理
BLE 通信	flutter_blue_plus	BLE Central 機能
機械学習	Core ML	推論・on-device 学習



図 3.2　Flutter 層のアーキテクチャ構成と Riverpod による状態管理

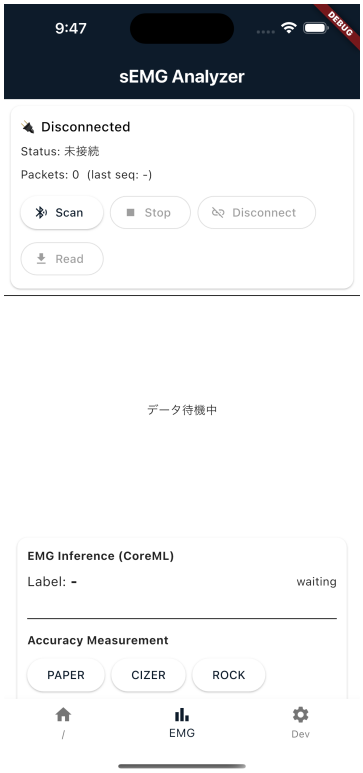


図 3.3　アプリケーションのアイドリング状態画面

3.3.2　実装した Service クラス群

データフロー上の各処理を Service クラスとして分離し,Riverpod Providers で依存性注入を行った. 以下に各 Service の役割を示す.

BleService（BLE 通信）

責務：Raspberry Pi Pico 2 W との BLE 通信の管理,sEMG データの受信, データ配信.

入出力：入力：BLE Notify による 42 バイトパケット出力：`BleDataPacket` の Stream

概要：`flutter_blue_plus` を用いて BLE Central 機能を実装し, デバイス名によるフィルタリング後に接続を行う. カスタム UUID のキャラクタリスティックを Notify 購読し, 受信したパケット（シーケンス番号 2B, サンプル 10 点 40B）をパースして Stream として配信する（図 3.4）.



図 3.4　BLE 通信フローとパケット構造

EmgWindowBuffer（データバッファリング）

責務：連続サンプル列を時系列ウィンドウに分割し, 特徴抽出処理の入力を生成する.

入出力：入力：連続する sEMG サンプル出力：150 サンプル長のウィンドウ

概要：内部バッファにサンプルを蓄積し, 150 サンプル（150ms）のウィンドウを 50 サンプル（50ms）ホップで切り出す. これにより 2/3 オーバーラップのウィンドウ系列を生成する（図 3.5）.

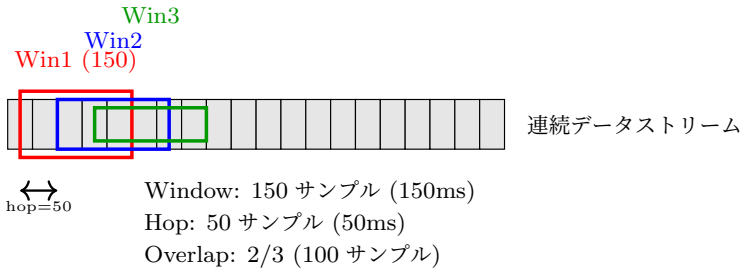


図 3.5　ウィンドウバッファリングによるデータ分割

EmgFeatureExtractor（特徴抽出）

責務：ウィンドウ化された sEMG 信号から分類用の特徴量を抽出する.

入出力：入力：150 サンプルの sEMG ウィンドウ出力：10 次元特徴ベクトル（正規化後）

概要：時間領域, 周波数領域, 線形予測の 3 カテゴリから構成される 10 次元特徴量を抽出する（表 3.3）. 抽出後,z-score 標準化を行い次段の推論・学習処理へ渡す.

表 3.3　抽出する 10 次元特徴量

カテゴリ	特徴量	次元数
時間領域	MAV, Variance, AW, Hjorth Complexity	4
周波数領域	CPS, LFO	2
線形予測	AR(4) 係数	4
合計		**10**

EmgMlService（推論・学習）

責務：Flutter 層と iOS Native 層を接続し, 推論および on-device 学習を実行する.

入出力：入力：10 次元特徴ベクトル出力：4 クラスの確率分布

概要：Method Channel を介して iOS Native 層の Core ML モデルを呼び出し, Neural Engine を用いた推論を実行する. また,`MLUpdateTask` を用いた on-device 学習を行い, 更新後のモデルを永続化する（図 3.6）.

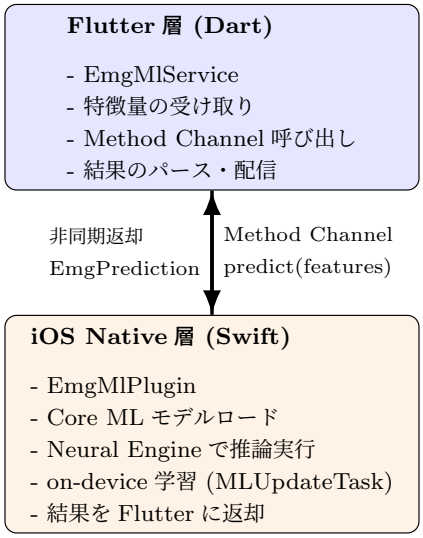


図 3.6　Flutter 層と iOS Native 層の推論フロー

EmgDatasetRecorder（データ記録）

責務：on-device 学習に用いるラベル付き特徴量データを管理・保存する.

入出力：入力：特徴量ベクトルおよびユーザ指定ラベル出力：{`features, classLabel, timestamp`} 形式のデータセット

概要：特徴量と対応ラベルをメモリ上に保持し, JSON 形式での保存および学習処理への受け渡しを行う. 収集データは `EmgMlService` の学習処理で直接利用できる形式とした.

共通処理パイプライン

推論およびデータ記録では, 以下の処理を共通パイプラインとして用いる.

1. BleService から受信したパケット Stream を購読する
2. EmgWindowBuffer で 150ms 窓を 50ms ホップで切り出す
3. EmgFeatureExtractor で 10 次元特徴量を抽出し正規化する
4. EmgMlService で推論（および必要に応じて学習）を実行する

3.3.3　用途（UI）ごとの機能

本節では,Service クラス群を用いて実装した主要機能を UI 単位で説明する.

リアルタイム推論（`EmgRealtimeInferencePanel`）

目的：受信した sEMG 信号をリアルタイムに分類し, 推論結果を可視化する.

使用 Service：BleService,EmgWindowBuffer,EmgFeatureExtractor,EmgMlService.

処理：共通処理パイプラインに従い,50ms 間隔で推論結果（4 クラス確率）を更新する.

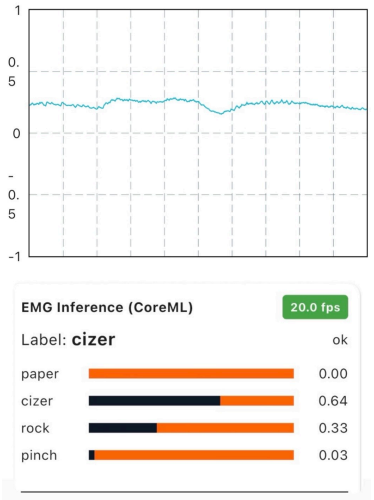


図 3.7　リアルタイム推論画面（波形表示と確率分布）

表示：sEMG 波形, 各クラスの確率分布, 最尤ラベルを表示する（図 3.7）.

精度測定（ラベル別 Top-1 正解率）

目的：特定ジェスチャ実行中の認識性能を簡易に定量化する.

使用 Service：BleService,EmgWindowBuffer,EmgFeatureExtractor,EmgMlService.

処理：ユーザが測定対象ラベルを選択し, 一定時間（3 秒間）推論結果を集計して, 測定対象ラベルと一致した割合（%）を算出する.

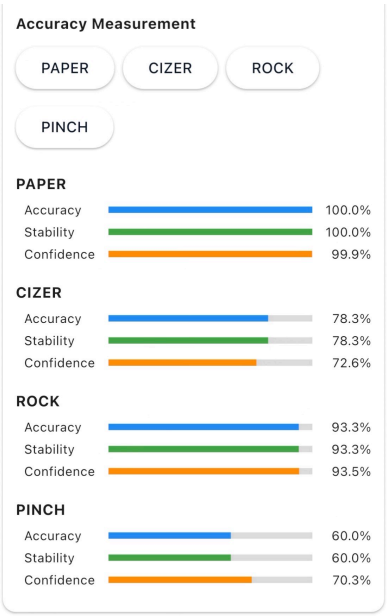


図 3.8　精度測定画面（ラベルごとの認識精度）

データセット記録（ラベル付き特徴量の収集）

目的：on-device 学習用のラベル付きデータセットを端末上で収集する．

使用 Service：BleService,EmgWindowBuffer,EmgFeatureExtractor,EmgDatasetRecorder.

処理：ユーザがジェスチャのラベルを指定して記録を開始すると，共通処理パイプラインの (1)〜(3) を実行し，特徴量にラベルと時刻を付与して蓄積する．

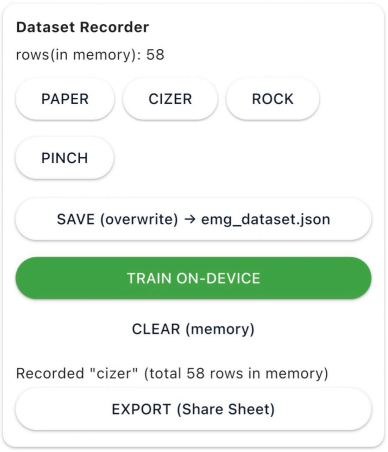


図 3.9　データセット記録画面（ラベル選択と保存・学習）

データの保存とエクスポート

収集データセットはローカルストレージへ JSON 形式で保存し，Share Sheet 経由で外部へエクスポートできる．オフライン解析や事前学習用データとしても利用可能である．

on-device 学習の実行

目的：収集データを用いて端末上でモデルを更新し，個人差への適応を行う．

使用 Service：EmgMlService,EmgDatasetRecorder.

処理：保存済みデータセットを学習用形式に変換し，Method Channel 経由で iOS Native 層へ渡して `MLUpdateTask` により学習を実行する．学習後のモデルは永続化され，次回起動時に自動ロードされる．学習前後の性能変化は精度測定機能（図 3.8）により確認する．

メモリクリア

`CLEAR（memory）`ボタンにより，メモリ上のデータセットを消去して新しいセッションを開始する．異なる条件でのデータ収集を繰り返し実行できる．

第 4 章

モデル設計および学習手法

4.1 特徴量設計

本研究では, 表面筋電位信号を 150 サンプル（1kHz で 150ms）の時間窓に分割し, 推論と特徴量計算にかかる Order を概算し, リアルタイム性を重視して 10 次元の特徴ベクトルを抽出する. 特徴量の内訳は, 時間領域 4 次元, 周波数領域 2 次元, 自己回帰モデル係数 4 次元である.

4.1.1 時間領域特徴量

時間領域特徴量として, 平均絶対値（MAV）, 分散, 平均波形長（AW）, Hjorth Complexity を用いる. 代表例として MAV は次式で定義される：

$$\mathrm{MAV} = \frac{1}{N}\sum_{i=1}^{N}|x_i| \tag{4.1}$$

Hjorth Complexity は信号の複雑さを表す指標であり, 本研究では標準的な定義に基づいて算出した（詳細は付録 A に示す）.

4.1.2 周波数領域特徴量

FFT により周波数スペクトルを算出し, スペクトル重心周波数（CPS）および低周波帯エネルギー比（LFO）を抽出する. 入力信号には Hann 窓を適用した.

4.1.3 自己回帰モデル係数

信号の時系列構造を捉えるため,4 次自己回帰（AR(4)）モデルを仮定し, Yule-Walker 法により係数 a_1〜a_4 を推定する. 導出および行列表現の詳細は付録 A に示す.

4.1.4　特徴量の正規化

抽出した特徴ベクトルは, 訓練データから算出した平均と標準偏差を用いて z-score 正規化を行う：

$$\tilde{f}_i = \frac{f_i - \mu_i}{\sigma_i + \epsilon}, \quad \epsilon = 10^{-6}. \tag{4.2}$$

4.2　ニューラルネットワーク構成

4.2.1　アーキテクチャ設計

本研究では, 軽量かつ高速な推論を実現するため,2 層の全結合ニューラルネットワーク（Multi-Layer Perceptron, MLP）を採用した. ネットワーク構成を図 4.1 に示す.

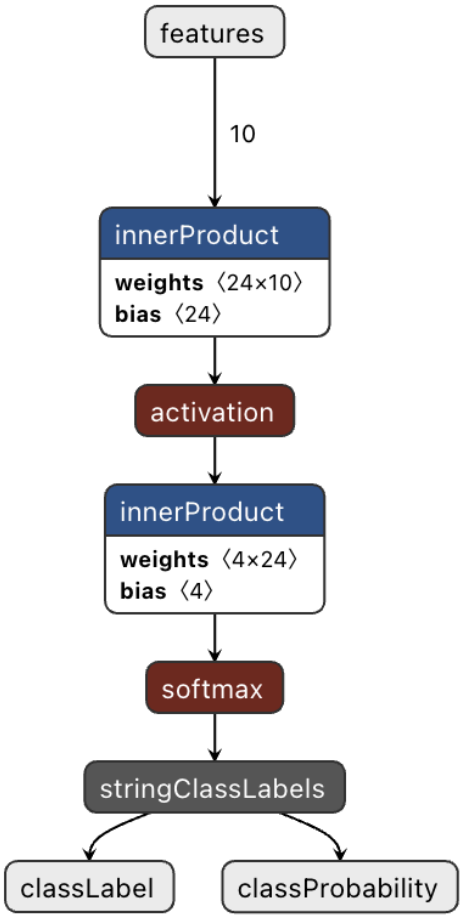


図 4.1　ニューラルネットワークアーキテクチャ（Core ML モデル）

4.2.2 全結合層採用の理由

本研究では, 入力が設計特徴量で構成されていることから, 畳み込み層ではなく全結合層を採用した. 主な理由を以下に示す：

- 入力特徴量には AR 係数を含む大域的な時系列情報が含まれ, 局所受容野を前提とする畳み込み層の利点を活かしにくい.
- 時間領域, 周波数領域, 自己回帰係数といった性質の異なる特徴量が混在しており, 次元間に空間的近接性が存在しない.
- 入力次元が 10 と小さく, 全結合層でも低い計算コストでリアルタイム推論を実現できる.

4.2.3 パラメータ数と計算量

各層のパラメータ数を表 4.1 に示す. 本モデルの総パラメータ数は 364, 順伝播に要する計算量は約 672 FLOPs であり, モバイルデバイス上でのリアルタイム推論に十分な軽量性を持つ.

表 4.1 各層のパラメータ数

層名	重み	バイアス	合計
隠れ層 $(W_1, \mathbf{b}_1)$	$24 \times 10 = 240$	24	264
出力層 $(W_2, \mathbf{b}_2)$	$4 \times 24 = 96$	4	100
総計			**364**

4.2.4 on-device 学習機能（Updatable Model）

本モデルは Core ML の Updatable Model 機能を用いて実装し, デバイス上での継続学習を可能とした. 出力層（$W_2, \mathbf{b}_2$）のみを更新対象とし, 隠れ層（$W_1, \mathbf{b}_1$）は固定した.

この構成により, 事前学習で獲得した特徴表現を保持したまま, ユーザー固有データに基づく分類境界の調整を行える. また, 更新対象パラメータ数は 100 個（総パラメータの 27.5%）に限定され, モバイル端末上での学習においても計算コストを抑制できる.

学習時の損失関数にはカテゴリカル交差エントロピーを用いる：

$$\mathcal{L} = -\sum_{i=1}^{4} y_i \log(p_i) \tag{4.3}$$

ここで y_i は正解ラベルの one-hot 表現,p_i はモデルが出力するクラス確率である.

第 5 章

実験および評価

5.1 実験条件

5.1.1 事前学習モデル

データ収集

アプリケーションのレコーダー機能を用い, 4 種類のジェスチャ（rock, paper, pinch, cizer）を各約 35 秒ずつ記録した. sEMG 信号は 150 サンプルの時間窓（150ms @ 1000Hz）, ホップ長 50 サンプル（重複率 66.7%）で処理し, 各窓から 10 次元の特徴ベクトルを抽出した.

データセット構成

表 5.1 に事前学習用データセットの構成を示す. 全クラスのサンプル数はほぼ均等であり, 抽出した全特徴ベクトルに対して z-score 正規化（$\epsilon = 10^{-6}$）を適用した.

表 5.1　事前学習用データセット

クラス	サンプル数	記録時間
rock	460	約 35 秒
paper	464	約 35 秒
pinch	462	約 35 秒
cizer	462	約 35 秒
合計	**1,848**	約 140 秒

訓練設定

事前学習はオフライン環境で実施し, 使用したハイパーパラメータを表 5.2 に示す.

5.1.2 実験手順

センサの装着状態により信号特性が変化する点を考慮し, **10 通りの装着条件**で実験を実施した. 各条件 i $(i = 1, \ldots, 10)$ において, 以下の手順を同一条件で実行した.

$$\text{学習前測定} \ \rightarrow \ \text{学習用データ収集} \ \rightarrow \ \text{on-device 学習} \ \rightarrow \ \text{学習後測定} \tag{5.1}$$

各装着条件の開始時には, モデルを事前学習状態にリセットし, 条件間の独立性を確保した.

各装着条件における手順

1. **学習前測定**：各ジェスチャを 3 秒間維持し, Accuracy,Stability,Confidence の 3 指標を測定する.
2. **学習用データ収集**：各ジェスチャを 3 秒間記録し, 特徴ベクトルを取得する（4 ラベル合計で約 200–240 サンプル）.
3. **on-device 学習**：収集したデータを用いて, 端末上で学習を実行する（表 5.2）.
4. **学習後測定**：学習前測定と同一条件で再度測定を行い, 各評価指標を算出する.

表 5.2　学習時のハイパーパラメータ

パラメータ	事前学習	on-device 学習
エポック数	300	10
バッチサイズ	32	16
学習率	0.05	1×10^{-3}
最適化手法	SGD	Adam
L2 正則化	1×10^{-4}	—
更新対象層	全層	出力層のみ
損失関数	カテゴリカル交差エントロピー	

5.2 評価指標

評価指標として,Accuracy,Stability,Confidence を用いる. いずれも各ジェスチャを 3 秒間維持した区間 T から算出した.

5.2.1 Accuracy

$$\text{Accuracy} = \frac{1}{|T|} \sum_{t \in T} \mathbb{1}[\hat{y}_t = y_{\text{true}}] \times 100 \tag{5.2}$$

5.2.2 Stability

測定区間における予測ラベルの最頻値 m の出現割合として定義する：

$$\text{Stability} = \frac{1}{|T|} \sum_{t \in T} \mathbb{1}[\hat{y}_t = m] \times 100, \quad m = \arg\max_{c} \sum_{t \in T} \mathbb{1}[\hat{y}_t = c]. \tag{5.3}$$

5.2.3 Confidence

$$\text{Confidence} = \frac{1}{|T|} \sum_{t \in T} \max_{c} p_{t,c} \times 100 \tag{5.4}$$



図 5.1　実験の様子（センサ装着とリアルタイム推論測定）

5.3　実験結果

5.3.1　10 条件における平均値

表 5.3 および表 5.4 に, 10 回の装着条件における各評価指標の平均値を示す.

表 5.3　学習前の評価指標（10 条件平均）

ラベル	Accuracy [%]	Stability [%]	Confidence [%]
paper	100	100	99.8
cizer	68.3	68.3	83.3
rock	97.2	97.2	90.4
pinch	45.8	46.9	66.1
平均	**77.8**	**78.1**	**84.9**

表 5.4　学習後の評価指標（10 条件平均）

ラベル	Accuracy [%]	Stability [%]	Confidence [%]
paper	96.0	96.0	94.6
cizer	76.3	76.3	75.6
rock	97.8	97.8	95.4
pinch	70.6	70.6	72.1
平均	**85.2**	**85.2**	**84.4**

5.3.2　学習前後の改善量

表 5.5 に,on-device 学習による各指標の改善量を示す.

表 5.5　on-device 学習による改善量（10 条件平均）

ラベル	ΔAccuracy [%pt]	ΔStability [%pt]	ΔConfidence [%pt]
paper	−4.0	−4.0	−5.2
cizer	+8.0	+8.0	−7.7
rock	+0.6	+0.6	+5.0
pinch	+24.8	+23.7	+6.0
平均	+7.3	+7.1	−0.5

改善量は学習後の値から学習前の値を減算したもの（Δ = 学習後 − 学習前）である. 正の値は性能向上, 負の値は性能低下を示す.

5.3.3　システムリソース

リアルタイム推論および on-device 学習時の CPU 使用率を計測した．測定には iPhone 15 を使用し, Xcode のデバッグツールによりプロセス単位の CPU 使用率を記録した．

測定条件

リアルタイム推論は, 窓長 150 サンプル（150ms）, ホップ長 50 サンプル（50ms）の設定で実行した．この設定により, 理論上の最大推論レートは 20fps（1000/50 = 20）となる．実測においても 20fps で安定して駆動することを確認した（図 5.2）．

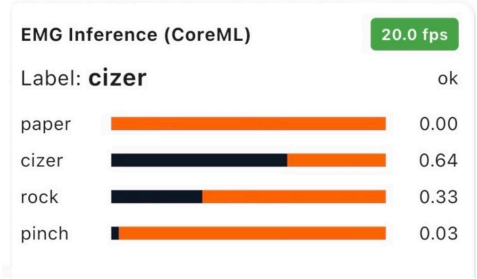


図 5.2　リアルタイム推論画面（20fps 駆動）

on-device 学習時間

on-device 学習にかかる時間を計測した．231 サンプルの学習データに対して, 学習時間は約 0.1 秒であった（図 5.3）．

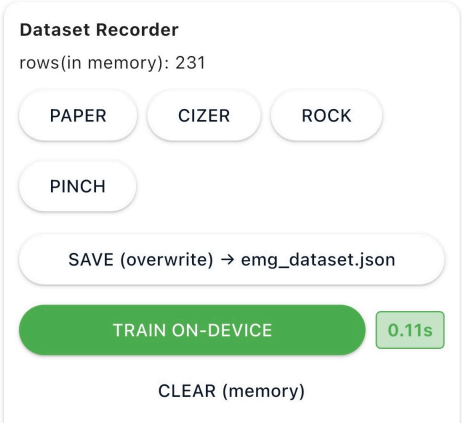


図 5.3　on-device 学習時間の計測画面（231 サンプル, 0.11 秒）

CPU 使用率

図 5.4 に各動作状態における CPU 使用率の推移を示す．表 5.6 に測定結果の概要を示す．

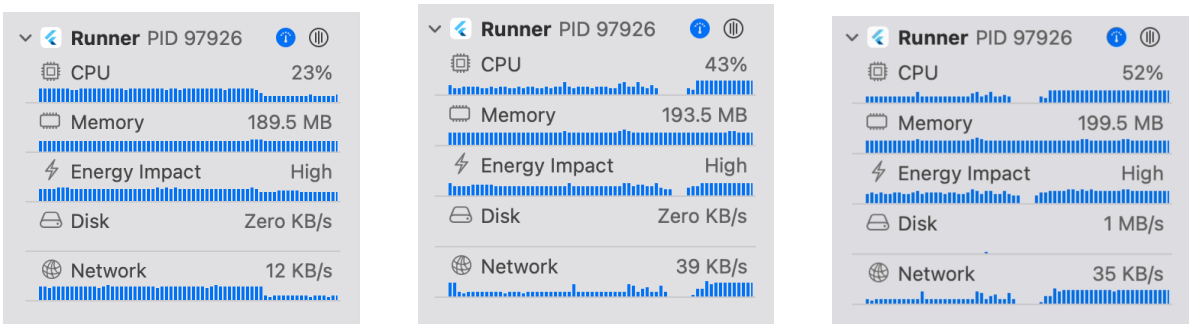


図 5.4　CPU 使用率の比較（左：アイドリング時，中央：リアルタイム推論時，右：on-device 学習時）

表 5.6　各動作状態における CPU 使用率

動作状態	CPU 使用率
アイドリング	約 20%
リアルタイム推論	約 40%
on-device 学習	約 50%

第 6 章

考察

6.1 結果の考察

6.1.1 on-device 学習による性能改善の特性

表 5.3, 表 5.4 および表 5.5 より, on-device 学習前後で以下の性能変化が確認された：

- Accuracy：77.8% → 85.2%（+7.3%pt）
- Stability：78.1% → 85.2%（+7.1%pt）
- Confidence：84.9% → 84.4%（-0.5%pt）

on-device 学習後に Accuracy および Stability が向上した一方で, Confidence はほぼ変化しなかった. 識別性能と予測の安定性が改善されつつ, モデルの確信度が過度に変化していないことを示している.

本研究では, 隠れ層を固定し出力層のみを更新する学習方式を採用した. この構成により,sEMG 特徴量が写像される特徴空間は保持されたまま, クラス間の判断境界のみが装着条件や個人差に応じて調整される. Confidence が学習前後でほぼ一定であったことは, 少量データによる学習であっても過学習を抑制し, 安定した適応が行われたことを示唆している.

クラス別に見ると, 事前学習時に識別精度が低かった pinch ジェスチャにおいて, 大きな性能改善が確認された. 一方で, 事前学習時点ですでに高精度であった paper ジェスチャでは, 判断境界の調整に伴うわずかな精度低下が見られた. 全体としては平均性能が向上しており, 識別が不安定であったクラスへの適応が有効に機能した.

以上より, 本研究で採用した最終層のみを更新する on-device 学習手法は, 確信度を維持したまま識別性能を改善できる点で, リアルタイム動作を前提とした sEMG インタフェースに適した特性を持つといえる.

6.1.2 アプリ性能

表 5.6 および図 5.4 より, 本システムのリソース消費について以下の特性が確認された. なお, 本実験では iPhone 15 を使用したため, 本モデルの推論および学習処理において Neural Engine が使用されておらず, すべての計算が CPU 上で実行されている. Neural Engine を活用可能な端末では, CPU 負荷のさらなる軽減や処理速度の向上が期待できる.

リアルタイム推論時の計算資源効率

リアルタイム推論時の CPU 使用率は約 40% であり, アイドリング時の約 20% と比較すると, 推論処理そのものによる追加負荷はおよそ 20% 程度に抑えられている. 20fps という推論レートは, ホップ長 50ms に基づく理論値どおりの動作であり, 推論周期に対して処理遅延がボトルネックとなっていないことを示している. この結果は, リアルタイム推論がシステム全体の計算資源を過度に消費することなく実現できていることを示唆しており, 限られた計算資源を持つモバイル端末上においても, 持続的な運用が可能であると評価できる.

on-device 学習による一時的負荷

on-device 学習時の CPU 使用率は約 50% であり, リアルタイム推論時と比較して約 10% の増加にとどまった. これは学習処理が推論処理に比べて計算量の増加を伴うものの, その影響が限定的であることを示している. 加えて, 学習時間自体は約 0.1 秒と極めて短いため, CPU 使用率の上昇は瞬間的なものであり, ユーザが体感する遅延や操作阻害につながる可能性は低い. このことから, 本システムにおける on-device 学習は, 実運用環境においても現実的な負荷水準で実行可能であると考えられる.

6.2 課題

本研究では on-device 学習を組み込んだシステム実装とその評価に注目したため, 単一被験者による評価に限られており, 被験者間差に対する一般性については検討できていない. また, 長時間使用時における学習効果の持続性や, データ量と学習回数の最適な関係についても, 今後の課題として残されている. さらに, 大規模データを用いた事前学習により特徴表現の質を向上させた場合の on-device 学習効果についても, 今後検討したい. 加えて, モデルの評価方法についても検討が必要であり, on-device 学習においてどの層を更新対象とすることでどのような評価指標の向上に寄与するかについても, 今後の検討課題である.

第 7 章

結論

本研究では, 単一チャネル sEMG を対象とした on-device 推論・学習システムを開発し, on-device 学習がリアルタイムジェスチャ検出の実用性能に与える影響を検討した.

まず, モバイル端末上でリアルタイムに動作する sEMG 解析システムを構築し, 特徴量抽出, 推論, データ記録, および学習を含む一連の処理を on-device で完結できることを示した. 軽量な ANN モデルと最終層のみを更新可能とする構成により, リアルタイム動作に十分な計算効率を確保した.

次に,on-device 学習の効果を実験的に評価した結果, Accuracy および Stability が向上する一方で, Confidence は学習前後でほぼ変化しないことが確認された. このことから, 本手法は予測の確信度を乱すことなく, 装着条件や個人差に起因する識別性能の低下を補正できることが示された. 特に, 事前学習時に識別が困難であったジェスチャに対して, 顕著な性能改善が得られた.

以上より, 本研究で開発した on-device 推論・学習システムは, 単一チャネルという制約条件のもとでも, リアルタイムジェスチャ検出において実用的な動的適応を実現できることを明らかにした.

参考文献

[1] M. Arozi, A. S. Prabuwono, and T. Herawan. Finger movement identification based on single channel semg using pca and ann. *Symmetry*, Vol. 12, No. 4, p. 541, 2020.

[2] A. Ibrahim, A. M. E. Safwat, O. M. Elzeki, and M. A. Al-Harthi. Single-channel semg hand gesture classification using an artificial neural network implemented on an esp32 microcontroller. *IEEE Access*, 2025.

[3] M. Zheng, M. S. Crouch, and M. S. Eggleston. Surface electromyography as a natural human–machine interface: A review. *IEEE Sensors Journal*, Vol. 22, No. 10, pp. 9198–9214, 2022.

謝辞

本研究を進めるにあたり, 研究の方向性や進め方について大きな裁量をお与えくださり, 自由な研究環境のもとでご指導くださいました安藤先生に, 心より深く感謝申し上げます.
また, 研究室の皆様のおかげで常に居心地の良い雰囲気の中で研究に取り組むことができました. 至らぬ点も多く, ご迷惑をおかけしましたが, 約半年間にわたり温かく支えていただきましたことに, 厚く御礼申し上げます. ここに感謝の意を表します.

大阪公立大学工業高等専門学校
総合工学システム学科
メカトロニクスコース
年澄 荘多

付録

付録 A：特徴量の詳細な数式定義

本研究で使用する 10 次元特徴ベクトルの詳細な数式定義を示す．各特徴量は 150 サンプル（サンプリング周波数 1000Hz で 150ms 相当）の時間窓から計算される．

時間領域特徴量

1. **平均絶対値 (MAV: Mean Absolute Value)**

$$\mathrm{MAV} = \frac{1}{N}\sum_{i=1}^{N}|x_i| \tag{.1}$$

2. **分散 (Variance / Hjorth Activity)**

$$\sigma^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i-\mu)^2 \tag{.2}$$

3. **平均波形長 (AW: Average Waveform Length)**

$$\mathrm{AW} = \frac{1}{N-1}\sum_{i=1}^{N-1}|x_{i+1}-x_i| \tag{.3}$$

4. **Hjorth Complexity**
   信号の複雑さを表す指標で, 以下のように定義される：

$$\mathrm{Mobility}(x) = \sqrt{\frac{\mathrm{Var}(\frac{dx}{dt})}{\mathrm{Var}(x)}} \tag{.4}$$

$$\mathrm{Complexity} = \frac{\mathrm{Mobility}(\frac{dx}{dt})}{\mathrm{Mobility}(x)} \tag{.5}$$

周波数領域特徴量

FFT（高速フーリエ変換）により周波数スペクトルを計算し, 以下の特徴量を抽出する．入力信号には Hann 窓を適用し, スペクトル漏れを抑制する．

5. **スペクトル重心周波数 (CPS: Center Power Spectrum)**

$$\mathrm{CPS} = \frac{\sum_{k=1}^{N/2} f_k \cdot P_k}{\sum_{k=1}^{N/2} P_k} \tag{.6}$$

ここで,f_k は周波数ビン,P_k はパワースペクトル密度を表す.

6. **低周波帯エネルギー比 (LFO: Low Frequency Ratio)**

$$\mathrm{LFO} = \frac{\sum_{f \leq 150\mathrm{Hz}} P_f}{\sum_{k=1}^{N/2} P_k} \tag{.7}$$

150Hz 以下の周波数帯域のパワーが全体に占める割合を表す.

自己回帰モデル係数

7. **AR 係数 (AR1–AR4)**

4 次自己回帰モデルの係数を Yule-Walker 法により推定する：

$$x_t = \sum_{j=1}^{4} a_j x_{t-j} + \epsilon_t \tag{.8}$$

自己相関関数 $r[k]$ を用いて、以下の Toeplitz 方程式を解く：

$$\begin{bmatrix} r[0] & r[1] & r[2] & r[3] \\ r[1] & r[0] & r[1] & r[2] \\ r[2] & r[1] & r[0] & r[1] \\ r[3] & r[2] & r[1] & r[0] \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} = \begin{bmatrix} r[1] \\ r[2] \\ r[3] \\ r[4] \end{bmatrix} \tag{.9}$$

これにより 4 つの係数 a_1, a_2, a_3, a_4 が得られる.